NUVRIQ AI

SDE 1 — Golang Backend

Pre-Interview Coding Assessment

TIME LIMIT	2 HOURS
PRIMARY LANGUAGE	GOLANG
DATABASE	POSTGRESQL
API	REST
DOMAIN	UNIFIED ENDPOINT MANAGEMENT
SUBMISSION	GITHUB REPOSITORY

Build a small device management and health service that can register devices, receive device heartbeats, identify inactive endpoints, and simulate a fleet of devices communicating with the service.

1. Why You Got This Task

At Nuvriq AI, we solve complex enterprise engineering problems at scale. This exercise evaluates how you approach a realistic component of a Unified Endpoint Management (UEM) system.

You will build a backend service for device inventory and health monitoring, together with a lightweight device simulator.

We are not looking for a complete UEM platform. A small, reliable implementation with clear engineering decisions is better than a broad unfinished solution.

What we want to understand

• Golang fundamentals and engineering practices

• REST API and HTTP understanding

• Database fundamentals

• Concurrency and reliability

• Problem-solving and edge-case reasoning

• Testing and debugging

• Ability to make sensible engineering trade-offs

2. The 2-Hour Task

Build a **Device Management & Health Service** using Golang.

The service should allow an enterprise administrator or device agent to register devices, update device synchronization state, retrieve devices, filter the fleet, identify inactive devices, and view a fleet health summary.

You should also create a lightweight **device simulator** capable of representing multiple devices communicating with the service concurrently.

3. Your Engineering Decisions

There is intentionally **no prescribed architecture or project structure** for this assessment.

You are free to decide:

• Application architecture

• Project and folder structure

• Database schema

• Repository/data-access approach

• HTTP framework or standard library

• PostgreSQL libraries

• Validation approach

• Concurrency model

• Testing strategy

• Docker/setup approach

We want to see how you make these decisions. Be prepared to explain **why** you chose your approach and what you would change if the system needed to support a significantly larger fleet.

4. Device Model

A managed device should contain enough information to identify it and determine its current health/synchronization state.

At minimum, the system should be able to represent:

• A unique device identifier

• Hostname or device name

• Platform

• OS version

• Associated user or owner

• Current status

• Last successful synchronization time

• Creation/update information where useful

You may choose the exact fields and data types.

5. Required API Capabilities

Implement REST endpoints that provide the following capabilities. The exact URL structure and request/response contracts are part of your design and should be documented.

5.1 Device Registration

A device should be able to register with the service.

The registration flow should validate input, establish a unique device identity, persist the device, and return an appropriate response.

5.2 Device Synchronization / Heartbeat

A registered device should be able to periodically communicate with the service and update its synchronization state.

The service should record when the device most recently synchronized successfully.

The endpoint must behave correctly when many devices send heartbeats concurrently.

5.3 Device Retrieval

An administrator should be able to retrieve information about an individual device.

The API should return an appropriate error when the requested device does not exist.

5.4 Device Listing and Filtering

An administrator should be able to retrieve a list of devices and filter the fleet by useful attributes such as platform and status.

The API should support pagination for larger device collections.

5.5 Inactive Device Detection

The service must provide a way to identify devices that have not synchronized within a configurable period.

For example, an administrator might request devices that have not synchronized for 60 days.

You do not need to wait 60 days during the assessment. Your sample data or simulator should allow this behavior to be demonstrated.

A device that has never synchronized must also be handled deliberately. Document your chosen behavior.

5.6 Fleet Summary

Provide a way to retrieve a useful summary of the device fleet.

The summary should contain enough information to understand overall device health, including total devices, synchronization/inactivity state, and platform distribution or other useful fleet-level information.

6. Device Client Simulator

UEM systems receive communication from many endpoint devices. Create a lightweight Go-based simulator that can represent multiple devices communicating with your service.

The simulator should be capable of running multiple simulated devices concurrently.

For example, the evaluator should be able to run something conceptually similar to:

```
go run ./... --devices=100
```

The exact command-line interface is up to you.

Simulator requirements

• Simulate multiple distinct devices.

• Allow devices to register and/or communicate with the backend.

• Send synchronization/heartbeat requests.

• Run multiple simulated devices concurrently.

• Allow some devices to stop communicating so inactivity can be demonstrated.

• Allow the evaluator to control the approximate number of simulated devices.

You may decide how the simulator is implemented and how it represents device state.

Simulator demonstration

Your sample scenario should make it possible to demonstrate a mixture of healthy, inactive, and never-synchronized devices without requiring real-world waiting periods.

7. Persistence

Use PostgreSQL for persistent device data.

You are responsible for deciding the schema and indexes that best support your API and query patterns.

Your design should account for the fact that a UEM system may eventually contain a large number of devices and frequent synchronization events.

Explain your important database decisions in the README.

8. Sample Data

Provide sample data sufficient to demonstrate the application. Include a mixture of:

• Windows, macOS, and Linux devices

• Recently synchronized devices

• Devices that have not synchronized for the configured inactivity period

• Devices that have never synchronized

Approximately 10–20 sample devices is sufficient.

9. Edge Cases

Consider and handle important edge cases. We are interested in your reasoning as much as the exact implementation.

• Device does not exist

• Invalid request data

• Duplicate device registration

• Unsupported platform

• Device has never synchronized

• Invalid or future synchronization timestamp

• Repeated synchronization requests

• Concurrent synchronization requests

• Database/API failures

10. Concurrency & Reliability

A UEM service may receive thousands of device heartbeats around the same time.

Your implementation should be safe under concurrent requests.

Think about:

• Concurrent device synchronization

• Shared application state

• Database consistency

• HTTP client behavior in the simulator

• Timeouts and failures

• Graceful shutdown

• What happens when the database is temporarily unavailable

You are free to choose the implementation strategy. We will discuss your reasoning during the interview.

11. API Quality

Use appropriate HTTP semantics, validation, status codes, and error responses.

The API should be reasonably easy for another engineer to understand and consume.

Do not commit credentials, passwords, API keys, or other secrets.

12. Testing

Include meaningful automated tests for important behavior.

You decide what level of unit and integration testing is appropriate within the two-hour limit.

At minimum, the evaluator should be able to see that you considered correctness for device creation, synchronization, inactivity detection, and invalid/not-found cases.

13. Docker & Local Setup

The application should be runnable locally.

Docker and Docker Compose are recommended for convenience, but the exact setup is your choice.

The README must contain clear, reproducible setup and execution instructions.

14. Optional Features

Optional features are intentionally open-ended. Only attempt them after the core requirements work.

• Bulk device registration

• Device search

• Device groups

• Synchronization history

• Device health scoring

• Metrics

• Rate limiting

• Retry/backoff behavior

• Additional fleet-management operations

Do not sacrifice required functionality for optional features.

15. Expected Demonstration

During the interview, you should be able to demonstrate an end-to-end flow covering:

• Registering a device

• Receiving a device synchronization/heartbeat

• Retrieving a device

• Filtering/listing the fleet

• Identifying inactive devices

• Viewing fleet summary information

• Running multiple simulated devices concurrently

• Demonstrating inactive/never-synchronized devices

The exact API design, command structure, and demonstration flow are up to you.

16. Definition of Done

■ Devices can be registered

■ Devices can send synchronization/heartbeat updates

■ Device information can be retrieved

■ Devices can be listed and filtered

■ Pagination is supported

■ Inactive devices can be identified

■ Fleet health summary can be retrieved

■ PostgreSQL is used for persistence

■ Important edge cases are handled

■ Concurrent sync requests are handled safely

■ A Go device simulator can simulate multiple devices

■ Automated tests are included

■ The application can be run locally

■ README explains how to run and evaluate the system

■ No secrets are committed

17. Evaluation Criteria

Area	Weight	What We Look For
Golang & Backend	30%	Go fundamentals, idiomatic implementation, HTTP handling, concurrency, error handling
API Design	20%	Clear contracts, validation, status codes, pagination, usability
Database	15%	Sound data model, queries, persistence, indexes, scalability considerations
Problem Solving	15%	Inactivity logic, edge cases, correctness, engineering judgment

Area	Weight	What We Look For
Architecture	10%	Clarity of design, separation of concerns, maintainability
Testing	5%	Meaningful tests and attention to correctness
Code Quality	5%	Readability, simplicity, maintainability

18. README Requirements

Your README should explain the solution you submitted.

• What you built and how it works

• Your architecture and why you chose it

• Your database model and important design decisions

• API documentation and examples

• How to run the backend, database, and simulator

• How to run the tests

• How inactive devices are determined

• How you handled concurrency and important edge cases

• What you intentionally simplified because of the two-hour limit

• What you would improve with another 4–8 hours

19. AI Tooling

AI coding tools are allowed and encouraged. You may use ChatGPT, Claude, Cursor, Codex, GitHub Copilot, Gemini, or similar tools.

Briefly document which AI tools you used and what you used them for.

You should also mention what you verified yourself and any important implementation decisions you changed.

You must be able to explain the code you submit.

20. What We Actually Evaluate

This exercise is not about building an entire UEM product in two hours.

We want to understand how you:

Understand → **Design** → **Implement** → **Test** → **Debug** → **Explain**

• Strong Golang fundamentals

• Ability to design an API from requirements

• Database understanding

• Concurrency and reliability awareness

• Ability to reason about edge cases

• Clean and maintainable implementation

• Debugging ability

• Engineering judgment

• Ability to make practical trade-offs

There is no single expected architecture. We care about the quality of the solution and your reasoning behind it.

Do not optimize for implementing every possible feature. Prioritize correctness, reliability, clarity, and a working end-to-end flow.

21. Submission

• Public GitHub repository

• README with complete setup and usage instructions

• Working Golang service

• PostgreSQL persistence

• Sample data

• Automated tests

• Device simulator

• Docker/Docker Compose if used by your solution

Do not commit API keys, passwords, production credentials, customer data, or personal data.

The submitted repository should represent the final version of your assessment.

NUVRIQ AI • SDE 1 ENGINEERING ASSESSMENT